

PYTHON

Module 22: Functions, Lambda & Logic

CODING

Modular Code: Python Functions

Functions are self-contained blocks of code that only run when called. At CodeHive, we use functions to maximize code reuse and modularity.

- ****Parameters vs Arguments****: A **parameter** is the variable listed in the function definition. An **argument** is the actual value sent to the function during a call.

- ****Return Values****: Use the `return statement to send data back to the caller. A function without a return statement implicitly returns None .`

Flexible Arguments: `*args` and `**kwargs`

When the number of inputs is unknown, Python offers 'Arbitrary Arguments'.

- **Arbitrary Arguments (`*args`)**: Collects positional arguments into a **tuple**. Use this when you don't know how many values will be passed.

- **Arbitrary Keyword Arguments (`**kwargs`)**: Collects named arguments into a **dictionary**. Use this for flexible configuration flags.

- **Unpacking**: You can use `*` and `**` during function calls to expand lists or dictionaries into individual arguments.

Scope & The LEGB Rule

Scope determines where a variable is accessible. Python follows the ****LEGB**** lookup order:

1. ****Local****: Inside the current function.
2. ****Enclosing****: Inside any nested/outer functions.
3. ****Global****: At the top level of the script.
4. ****Built-in****: Python's pre-defined names.

****Keywords****: Use `global` to modify a global variable from a local scope, and `nonlocal` for variables in nested function scopes.

Lambda: The Anonymous Function

A

lambda function is a small, one-line function without a name. It can take any number of arguments but can only contain a single expression.

- **Syntax**: `lambda arguments : expression`
-

- **Usage**: Best used as temporary functions inside higher-order functions like `map()`, `filter()`, or `sorted()`.
-

Decorators: Functional Wrapping

Decorators allow you to add behavior to an existing function without modifying its source code. They 'wrap' the original function and are applied using the `@decorator_name` syntax.

- ****Metadata****: Use `functools.wraps` to ensure the original function's name and docstring are preserved after decoration.

Recursion: The Self-Calling Logic

Recursion is a programming technique where a function calls itself to solve smaller instances of the same problem.

- ****The Base Case****: Essential for stopping the recursion. Without it, you will trigger a **RecursionError (Stack Overflow)**.

- ****Performance****: While elegant, recursion has a depth limit (default ~1000). For heavy processing, iteration is often more efficient.